

Workshop - Spec Coding with IBM Open Source Data



WiFi – IBMGuest

Blank-5-Snarl-4-Avoid

Workshop - Spec Coding with IBM Open Source Data

11:30	Registration, Lunch, Intro, Database Setup
13:00	Spec -> Design -> contract-first loop
14:00	Tasks -> Implementation
15:00	Add customizations / iterate via Spec / Agentic AI
15:30	Project presentations

```
Command Prompt
File Edit View Terminal Tabs Help

$ whoami
Aaron Ploetz
developer advocate
DataStax / IBM

author:
• Seven NoSQL Databases in a Week
• Mastering Apache Cassandra 3.x
• Code with Java 21
• Think Like a Programmer

committer:
Apache Cassandra® project

engineer:
Grainger, AccuLynx, Target
```



Spec first, code second.

Write detailed specifications — requirements, data models, API contracts — and use them as executable blueprints for an AI coding agent. The spec is your source of truth. The agent generates the code.

THE KEY INSIGHT

Planning matters even more with an agent. Iterate on the plan first, then hand it off. The spec is the first artifact you and the AI build together.

NOT VIBE CODING

No “just make it work” prompting.
You define what right looks like
before any code is written.

NOT WATERFALL

Each step is iterative. Refine requirements, sharpen the contract, adjust tasks — as many loops as you need.

THIS WORKSHOP

Three steps: requirements, spec (design + OpenAPI), then tasks + code. You’ll build a real app using this workflow.

“Spec-driven development is one of the most important practices to emerge in 2025.” — Thoughtworks Technology Radar

Guidelines:

Keep manual steps to a minimum!

Anything you need to do on the command line, your agentic IDE should be able to do for you.

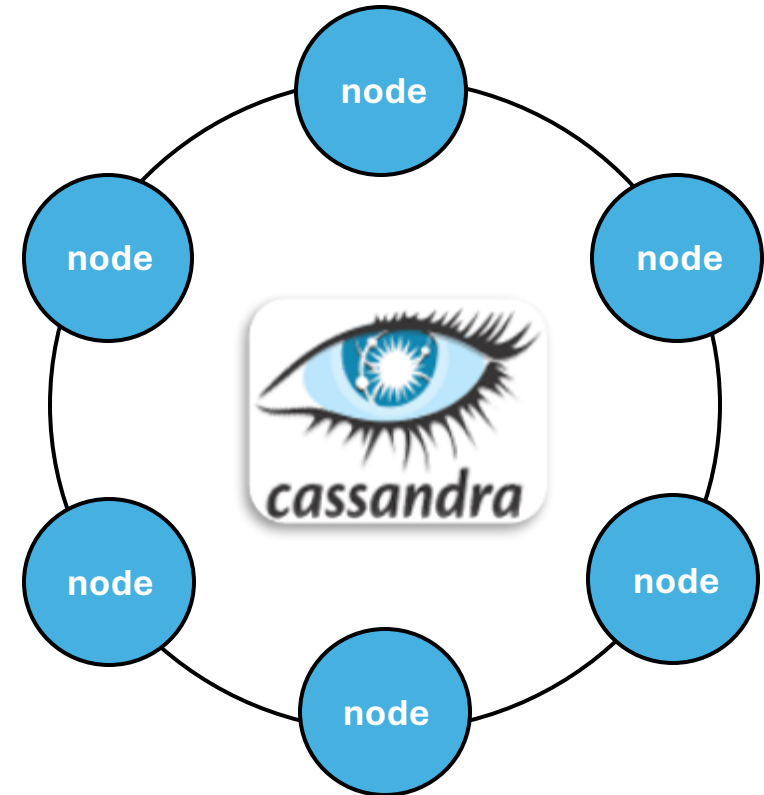
Contract before code.

Build your requirements, design, and api spec. Then generate a to-do list...*then* build the code.

What is Cassandra?

Apache Cassandra®

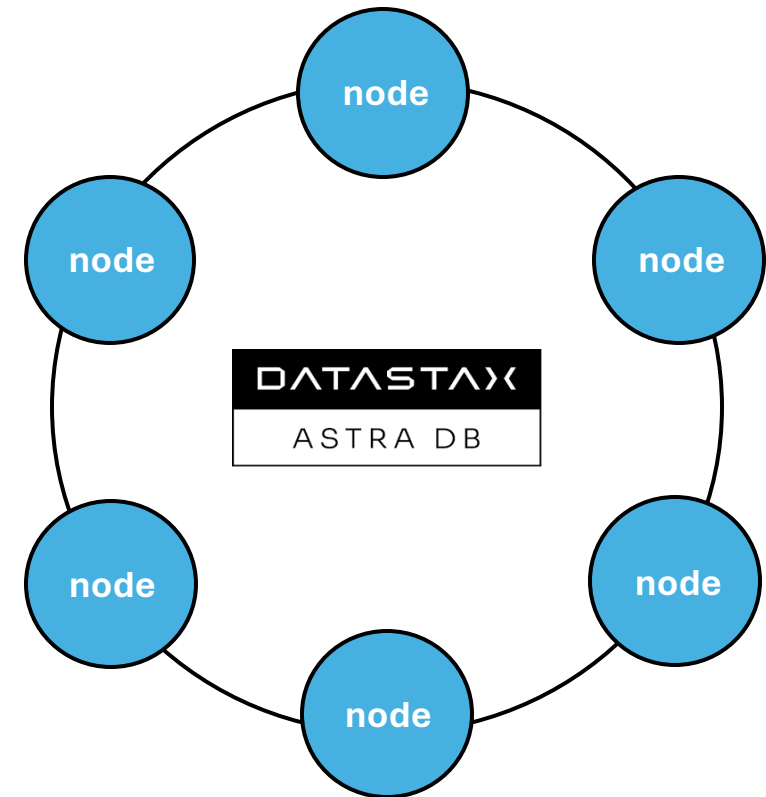
- Highly Available NoSQL database (AP).
- Geographic distribution.
- Scalable read and write performance.
- Designed to run on commodity hardware.
- All nodes are peers.



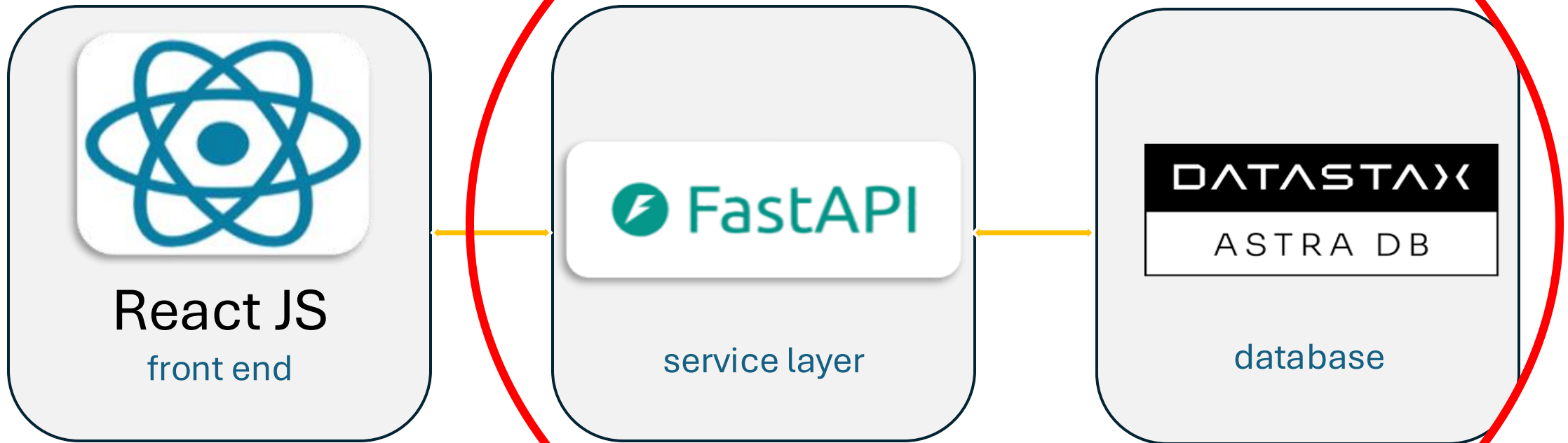
What is Astra DB?

Astra DB

- Cloud native, managed service built on **Apache Cassandra®**.
- No operational overhead.
- Runs in all 3 major clouds.
- “No touch” Scalability.



Application stack for today



Bonus

Astronaut & Spaceflight Tracker

```
{  
  "name": "Jim Lovell",  
  "dob": "1928-03-25",  
  "birthplace": "Cleveland, OH",  
  "university_name": "US Naval Academy"  
}
```

```
{  
  "items": [  
    {  
      "mission_name": "Apollo 11",  
      "astronaut_name": "Buzz Aldrin",  
      "ship_name": "Columbia",  
      "mission_start_date": "1969-07-16T18:32:00Z",  
      "mission_end_date": "1969-07-24T21:50:35Z"  
    },  
    {  
      "mission_name": "Apollo 11",  
      "astronaut_name": "Michael Collins",  
      "ship_name": "Columbia",  
      "mission_start_date": "1969-07-16T18:32:00Z",  
      "mission_end_date": "1969-07-24T21:50:35Z"  
    },  
    {  
      "mission_name": "Apollo 11",  
      "astronaut_name": "Neil Armstrong",  
      "ship_name": "Columbia",  
      "mission_start_date": "1969-07-16T18:32:00Z",  
      "mission_end_date": "1969-07-24T21:50:35Z"  
    }  
  ],  
  "count": 3  
}
```



Astronaut & Spaceflight Tracker

- Stores data on individual astronauts.
- Stores data on spaceflight missions.
- Allows CRUD operations on both.
- Serves restful endpoints to each underlying operation.



↓ Clone this GitHub repository ↓

```
git clone git@github.com:aar0np/spec_coding_astronaut_app.git
```

Astronaut & Spaceflight Tracker

Pre-requisites

- Python 3.10 or 3.11
- An Astra DB account
- An agentic coding tool (Cursor, Claude Code, IBM Bob)



↓ Clone this GitHub repository ↓

```
git clone git@github.com:aar0np/spec_coding_astronaut_app.git
```

Exercise #1

Create an Astra DB Database

<https://astra.datastax.com>

Create an Astra DB Database

Click the "Create database" button (upper right corner) to create a new Astra DB instance:

- Select "Serverless (Vector)"
- name: *whatever*
- Pick a cloud provider.
- Select a "free" region.

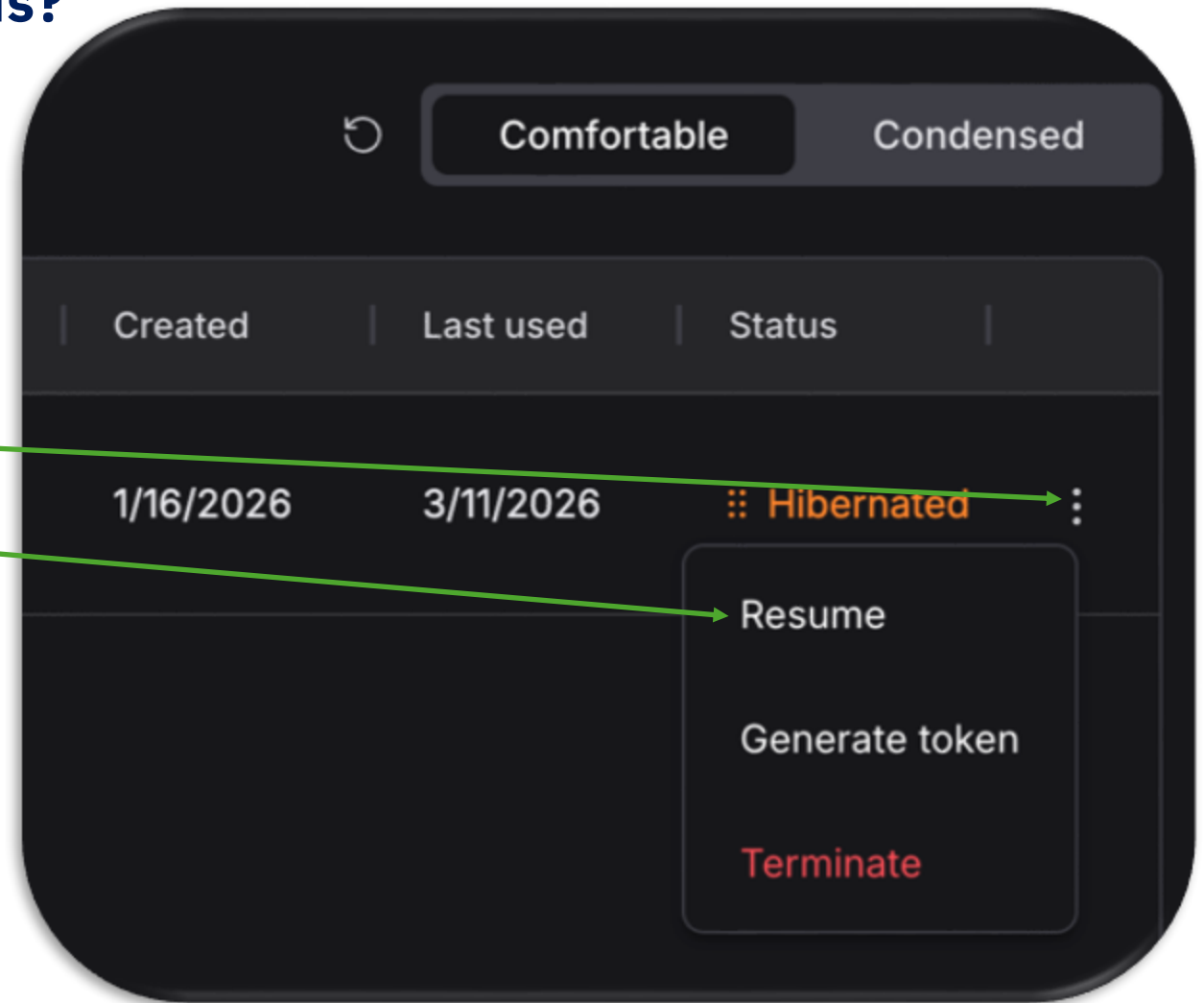
The screenshot shows the 'Create database' modal window. At the top right, there are 'ESC' and 'X' icons. Below the title, there are two selection boxes. The first box, 'Serverless (vector)', is selected with a checkmark and contains the text 'An all-in-one database solution, optimized for Vector and Generative AI workloads'. The second box, 'Serverless (non-vector)', contains the text 'A more traditional database solution without any of our new vector capabilities'. Below these is a 'Database name' field with a red asterisk, containing the text 'aaronfdb'. A note below the field says 'Give it a memorable name – this can't be changed later.' Under the 'Provider' section, there are three buttons: 'Google Cloud' (selected), 'Amazon Web Services', and 'Microsoft Azure'. Below that is a 'Region' dropdown menu with a red asterisk, showing 'us-east1' with a flag icon. At the bottom right, there are two buttons: 'Cancel' and 'Create database'.

<https://astra.datastax.com>

Is your database in a "**Hibernated**" status?

To resume an Astra DB Database

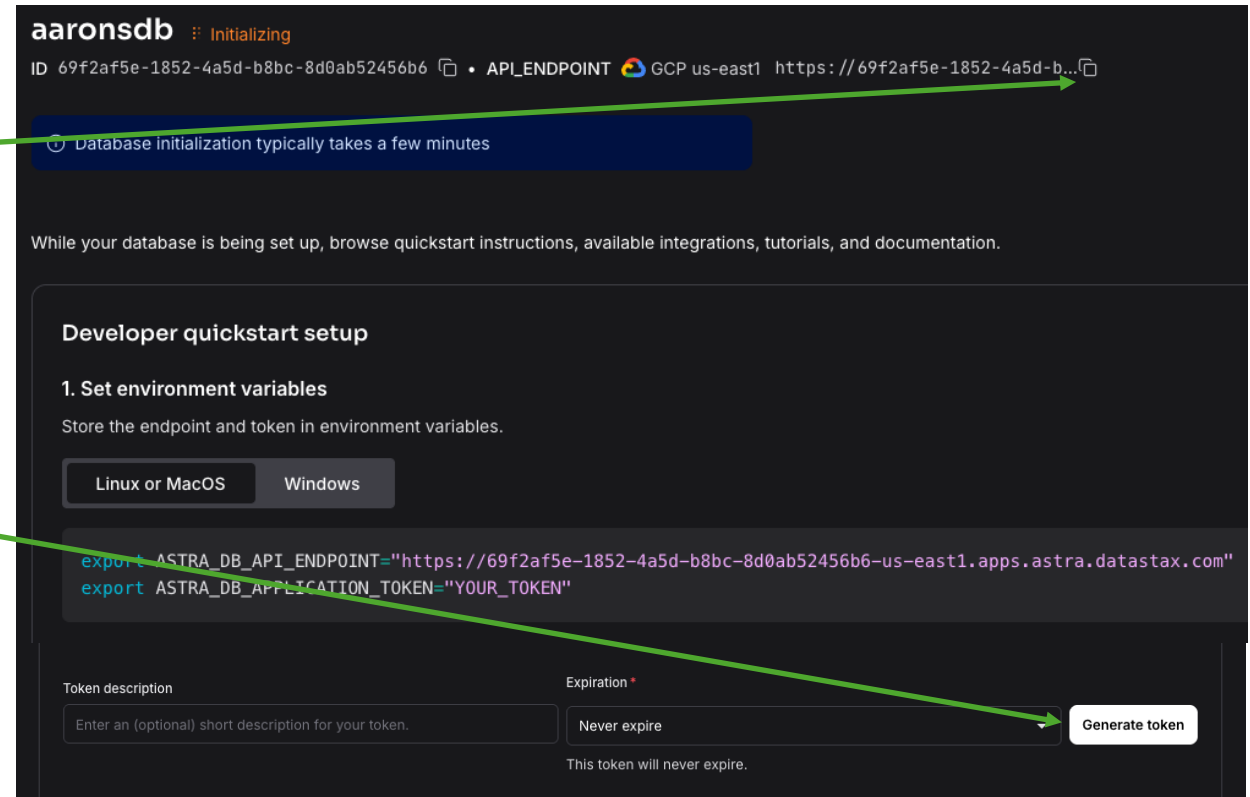
- Click the "3 dots" menu
- Select "**resume**"



<https://astra.datastax.com>

Create your credentials

- Once available, copy the **API_ENDPOINT** by clicking its copy button.
- Click the "Generate token" button (lower right corner) to create a new access token.

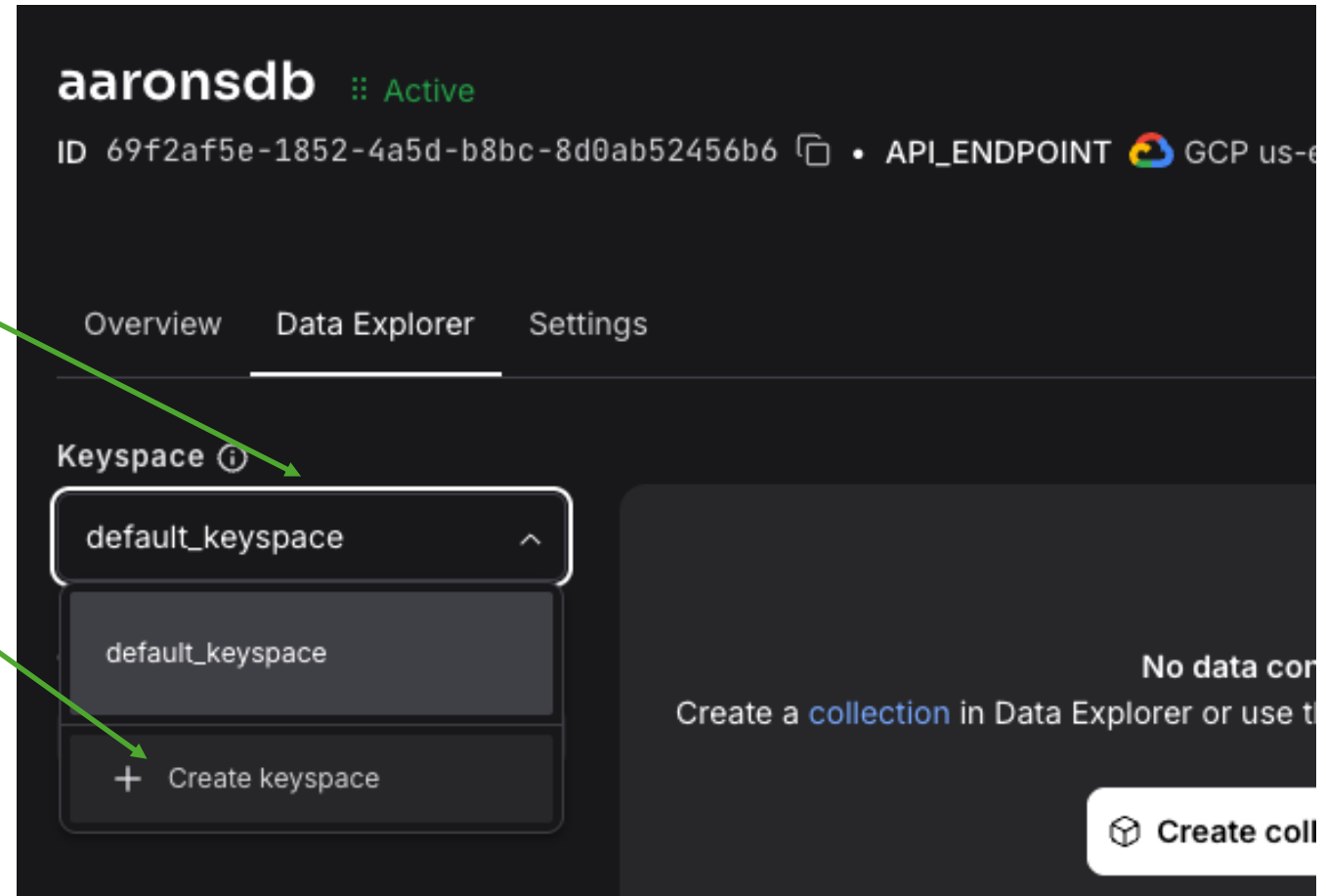


Be sure to save the endpoint and token for later!

<https://astra.datastax.com>

Create a keyspace

- Click the "Keyspace" menu.
- Select "Create keyspace" and name it "astronaut_app"



DO NOT click "Create collection!"

Exercise #2

Create schema

Astronaut & Spaceflight Tracker Data Model



```
CREATE TABLE astronauts (  
  name TEXT PRIMARY KEY,  
  dob DATE,  
  birthplace TEXT,  
  university_name TEXT);
```

```
CREATE TABLE astronauts_by_mission (  
  astronaut_name TEXT,  
  ship_name TEXT,  
  mission_name TEXT,  
  mission_start_date TIMESTAMP,  
  mission_end_date TIMESTAMP,  
  PRIMARY KEY (mission_name, astronaut_name);
```

DATASTACK

ASTRA DB

Exercise #3

Create environment file

Install dependencies – command line

```
pip install astrapy==2.2.1  
pip install poetry
```



Install dependencies - agent conversation

```
Please install the poetry and astrapy==2.2.1  
Python packages for me.
```

Copy environment example file to `.env`.

```
cp .env.example .env
```

Or -

Can you rename the `.env.example` file to `.env`, and open it so I can add my credentials?

Edit the `.env` file, adding your credentials.

```
ASTRA_DB_API_ENDPOINT=https://...  
ASTRA_DB_APPLICATION_TOKEN=AstraCS:...  
ASTRA_DB_KEYSPACE=astronaut_app
```



Exercise #4

Load data

Astronaut & Spaceflight Tracker

- Data loaders in the `loaders/` directory.
- CSV files in the `data/` directory.
- Both loaders import the `app/core/config.py` module.
- Be sure to set the credential variables in the `.env` file!



```
Please run the load_astronauts.py data loader for me.
```

```
Please run the load_astronauts_by_mission.py data loader for me.
```

CQL commands to try:

```
use astronaut_app;
```

```
desc table astronauts_by_mission;
```

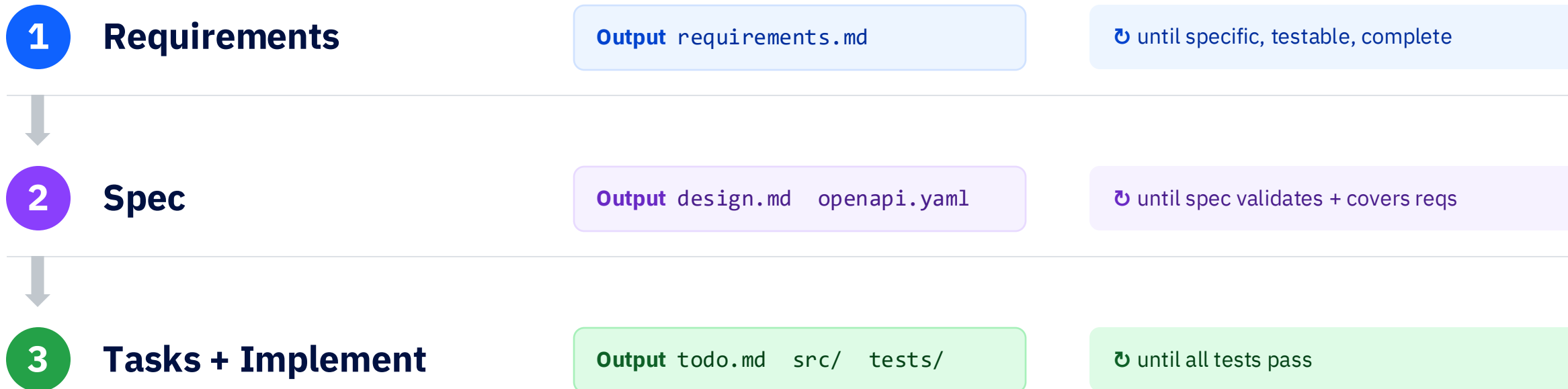
```
SELECT * FROM astronauts_by_mission LIMIT 5;
```

 CQL console



Building the Service Layer

Three steps. Iterate each one until it's right.



What “iterate” looks like

Prompt the agent. Review the output. Refine the prompt. Run again.

Don't move to the next step until you're happy with the current output.

The spec files are your ground truth — they survive every iteration.

Goal + Context + Constraints + Examples = Awesome Prompt

Goal

What you want built/changed (definition of done). **Bonus:** Defined Role

Context

Stack + relevant code/files

Constraints

Must/never rules (deps, API, perf, style)

Examples

Inputs → outputs + edge cases

Pro Tip: Have the agent interview you. “You are a product manager. Interview me about X”

Context is everything

Better context = fewer wrong assumptions, less refactoring, faster correct output

Include only what matters:

- The relevant files / interfaces
- Docs
- Websites

MCP

Avoid if possible

Contributes to context bloat

Context7 MCP maybe an exception

Skills

Small bundles of docs + instructions

Can dynamically load MCPs

Easy to make

<https://docs.datastax.com/en/astra-cli/index.html>

<https://docs.datastax.com/en/astra-db-serverless/api-reference/dataapiclient.html>

Pro Tip: Have the agent create a doc directory and scrape docs to create local docs on everything

WHAT WE'RE DOING

Write a clear set of requirements for your project. Think user stories, acceptance criteria, and constraints. Give each requirement a testable ID (e.g. REQ-001). Avoid anything technical!

HOW TO PROMPT



```
# Tell the agent what you're building

I'm building a [your project]. Write requirements in requirements.md with:

- Numbered IDs (REQ-001, REQ-002 ...)
- Acceptance criteria for each
- Personas and User flows
- Do not include any implementation details such as code or technology choices
```

DONE =

requirements.md exists with numbered, testable requirements that you've reviewed and are happy with.

Pro Tip: Check for scope creep. That happens. A LOT

Finished early? Ask the agent to add edge cases or “what could go wrong” scenarios.

Make sure you are in the correct "mode."

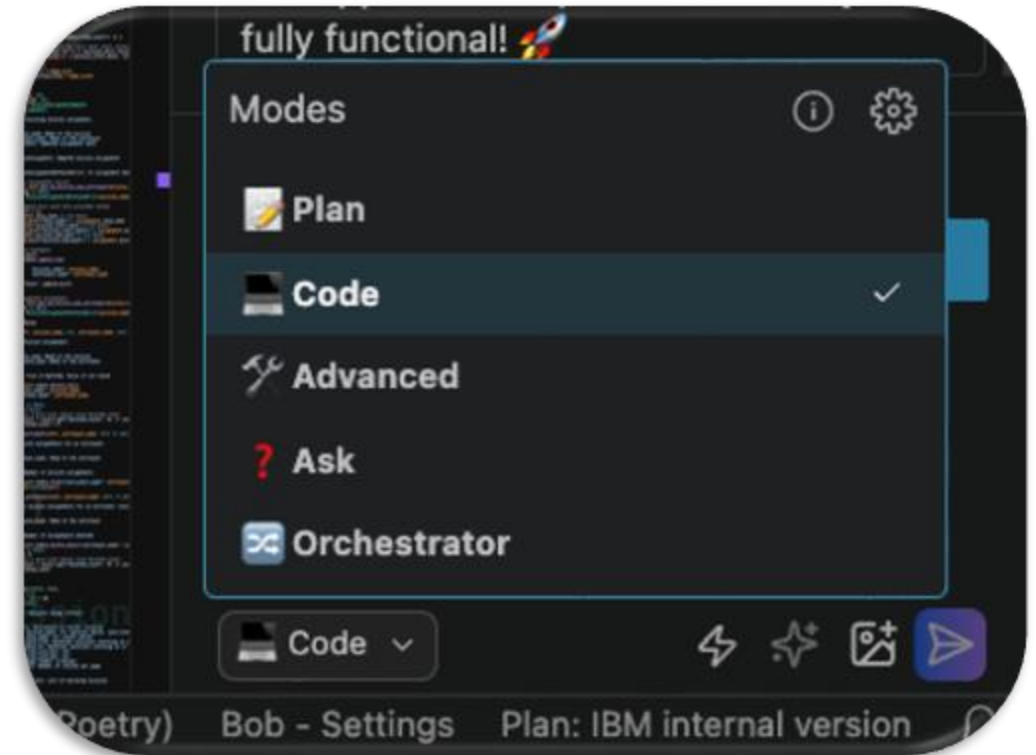


IBM Bob has different modes to govern its output.

Specifically, pay attention to these two:

- **Code**
- **Plan**

We will need to be in "Plan" mode until we start following the todo list!



Exercise #5a

Building the requirements.md file

What you should have now

Your repo

- ✓ requirements.md
- ✓ .env
- ✓ .gitignore

SAMPLE REFINEMENT PROMPT



```
Review requirements.md. Are there  
any gaps, ambiguities, or missing  
edge cases? Fix them and update  
the file.
```

Next up → Step 2: Spec (design.md + openapi.yaml)

WHAT WE'RE DOING

Turn your requirements into a data model (design.md) and an OpenAPI contract (openapi.yaml). The OpenAPI file becomes the single source of truth for your API.

HOW TO PROMPT



```
# Feed it the requirements as context
```

```
Read requirements.md. From those requirements:
```

1. Create design.md with data model + collections, web frontend, and backend service layer which uses Astra DB.
2. Create openapi.yaml (OpenAPI 3.1) with
all endpoints, schemas, and error responses
3. Map each endpoint back to a REQ-ID

DONE =

design.md + openapi.yaml that validates and maps every requirement to an endpoint.

Finished early? Ask the agent to validate the OpenAPI spec and add example responses for each endpoint.

Exercise #5b

Building the design.md and OpenAPI.yaml files

What you should have now

Your repo

- ✓ requirements.md
- ✓ design.md (new)
- ✓ openapi.yaml (new)
- ✓ .env
- ✓ .gitignore

SAMPLE REFINEMENT PROMPT



```
Validate openapi.yaml. Check that  
every REQ-ID in requirements.md is  
covered by at least one endpoint.  
List any gaps and fix them.
```

Next up → Step 3: Tasks + Implement (todo.md + working code)

Exercise #5c

Building the todo.md file

WHAT WE'RE DOING

Break the spec into a task list (todo.md), then have the agent implement each task: server, frontend, Astra DB integration, and tests. Run tests after each task.

HOW TO PROMPT



```
# Give it all the context it needs
```

```
Read requirements.md, design.md, openapi.yaml.
```

1. Create todo.md breaking the spec into tasks
2. Implement each task. Use .env for creds.
3. Write tests and run them after each task
4. Mark each task done in todo.md when it passes

DONE =

A working server + frontend connected to Astra DB. All tests passing. Every task checked off in todo.md.

Pro Tip: Use github issues instead of a file. Using the gh command line tool is extra bonus tip

Finished early? Ask the agent to add error handling, improve the UI, or write integration tests against Astra.

What you should have now

Your repo

- ✓ requirements.md
- ✓ design.md
- ✓ openapi.yaml
- ✓ **todo.md** (new)
- ✓ **src/** **tests/** (new)

SAMPLE REFINEMENT PROMPT



Run all tests. For any that fail:

1. Read the error output
2. Check openapi.yaml for the expected behavior
3. Fix the implementation, not the test
4. Re-run until green

You did it! A working app, a reusable workflow, and a repo full of proof — built with spec-driven development.

Exercise #5d

Execute each TODO task

Switch to "code" or "agent" mode.

Try testing some of the endpoints

```
curl -X GET -s "http://localhost:8000/astronauts/Pete%20Conrad" | jq  
{  
  "name": "Pete Conrad",  
  "dob": "1930-05-02",  
  "birthplace": "Philadelphia, PA",  
  "university_name": "Princeton University"  
}
```



```
curl -X POST -s "http://localhost:8000/astronauts" \  
-H "Content-Type: application/json" \  
-d '{ "name": "Victor Glover", "dob": "1976-04-30",  
"birthplace": "Pomana, CA", "university_name":  
"California Polytechnic State University" }'
```



Your specific endpoints and payloads may be different!

(Bonus)

Exercise #6

Build a simple web UI

What else could you add?

Endpoints for...

- Equipment tracking
- Payload management
- Mission planning

Or build a web front end to serve the data?



Thank you!

